

Firefly

Pitch and checkpoint plan · East v. West AllStar Hack · Defense track · Dean + Chase

Saturday Sept 12, 12:00 PM ET → Monday Sept 14, 1:00 PM ET · 49 hours · five scored check-ins plus the final

The pitch

When a structure catches fire, nobody knows where the fire is. We build the system that figures it out and sends the drones.

Firefly is the decision brain for a firefighting drone swarm that lives inside large enclosed structures — high-rises, apartment blocks, warehouses, hangars, parking structures, data centers, ships. It estimates the state of a fire from sensors the fire is actively destroying, and directs a heterogeneous fleet to contain that fire until human crews arrive. It is a tool for firefighters, not a replacement for them.

Scope: broad problem, one demo

The problem is general and the market is every large occupied structure. The demo is a Navy amphibious assault ship, for three reasons: the failure case is documented and severe, the correlated-sensor-failure story is at its cleanest in a steel hull, and the Defense track explicitly covers dual-use work. Nothing in the system is naval — the structure is data, not code.

The problem

Layer one: the obvious problem

Fire inside a large enclosed structure spreads faster than people can reach it. High-rise crews climb dozens of flights carrying equipment, arrive exhausted, and have almost no picture of conditions above them. At sea it is worse: the USS Bonhomme Richard caught fire in port in 2020, burned for four days, and was scrapped, with its suppression systems tagged out for maintenance. In both cases the first responders enter nearly blind.

Layer two: why a swarm

You want something already inside the structure that starts working in second one, not minute fifteen. Tethered units on the standpipe or water main holding a line. Retardant carriers coating unburned material ahead of the fire. Units closing doors to starve it of oxygen. Scouts that are pure sensors. In a high-rise, units can also enter through windows and reach the fortieth floor in ninety seconds, which no ground crew can match. They do not extinguish. They contain, and they buy the crew time.

Layer three: the actual hard problem

A swarm is worthless if it does not know where the fire is — and knowing that is far harder than it sounds, because **the fire destroys the sensors meant to report it**. Smoke blinds thermal cameras so they read cold where it is hottest. Heat cooks sensors until they saturate. Comms cut and a reading freezes at its last value while still appearing current. And they do not fail one at a time: a space flashes over and every sensor inside it dies in the same instant.

This is not noise. Noise is small, random, and averages out. This is **corruption** — readings wrong in structured, coordinated ways. The textbook tools assume independent noise, and under correlated corruption they do not merely get fuzzy. They converge *confidently on the wrong answer*. Which means drones dispatched to the wrong floor while the real fire spreads.

The solution

Two pieces that turn out to be one piece.

An estimator that knows what it does not know

- **Physics as a trusted reference.** Fire cannot exist in a space with no heat path to a burning one, so a sensor reporting flames in an isolated space is provably broken rather than reporting a new fire. A consistency check grounded in conservation, not statistics — it cannot be fooled by a coordinated group of sensors all lying the same way.
- **Separating "hot here" from "burning here."** Heat and smoke travel far from their source: through steel in a hull, up shafts and stairwells in a tower. A sensor reading badly on floor 30 may be reporting a fire on floor 22. Different claims, modeled differently.
- **Honest ambiguity.** When surviving readings are consistent with two fire states, it reports both and lowers confidence rather than picking one. This sounds like a weakness and is the core feature: a wrong confident answer sends drones to the wrong floor, while "floor 22 or 30, covering both" is what an experienced incident commander actually says.

An allocator that treats drones as sensors

Every drone sent somewhere is both a suppression asset and a measurement, so where we choose to fight determines what we can see next tick. Suppression and observability normally live in separate literatures. On a swarm they are the same decision, and we solve them jointly.

Why this is the Defense track. The Playbook covers "national security, defense, or **dual-use** applications." The challenge brief asks for an estimator that preserves useful knowledge of a critical system under a defined class of measurement and communication failures, where the decision-maker needs both the estimate and whether it is supported by the surviving observations. Its FAQ states plainly that a military application is not required and that infrastructure resilience and emergency operations qualify.

Structure as data, not code

The generality must be in the schema from hour zero, or it becomes a three-directory refactor at hour 30. Nothing in the type system names a ship.

```
type Space = {
  id: string;   level: number;           // deck, floor, storey
  neighbors: string[];                  // same-level adjacency
  above: string|null; below: string|null; // vertical coupling
  temp: number; burning: boolean; fuel: number;
  hazard: 'none'|'ordnance'|'fuel'|'chemical';
  occupants: number; doorsOpen: string[];
};
```

A structure plan is a JSON file: levels, spaces, adjacency, vertical coupling, hazards, occupants, riser and resupply points, and fixed sensor locations. Swapping ship for tower is a different file, not different code. The one genuine physics difference is the dominant spread path — conduction through steel in a hull, stack effect up shafts in a tower — so the plan carries per-edge transfer rates rather than hardcoding one mechanism.

	Ship (demo)	High-rise (stretch)
Dominant spread	Conduction through steel; fire moves without flame passing	Stack effect up shafts and stairwells; window failure vents
Trusted sensors	Hardwired damage-control sensors	Existing alarm panel, smoke detectors, sprinkler flow switches
Access	Interior only	Interior plus exterior through windows
Occupants	Known, trained crew	Unknown count, untrained, self-evacuating

What exists at the end

1. **A working system.** A structure, a fire spreading across levels, a swarm fighting it, and a split view of ground truth beside what the brain believes. Judges can break things live and watch it recover.
2. **A head-to-head result.** Our estimator against a competent Kalman baseline on identical seeds and identical corruption, scored on false certainty, estimation error, time to recovery, and compute cost — the four metrics the brief names.
3. **A negative result.** A specific pair of fire states our sensor set provably cannot distinguish, the operational cost, and the one extra observation that resolves it. The brief says a proof of impossibility can be a strong result, and few teams will bring one.
4. **A repo and writeup.** Question, contribution, assumptions, limits, next test.

Market framing for the business-value slide: ships and hangars, then high-rises, apartment blocks, warehouses, parking structures, and data centers. Keep the pitch line singular even so — a team that solved one hard problem and noticed it generalizes scores better than a platform for everything.

Hold onto this: **the demo is not the deliverable, the result is**. The simulation exists to make the result legible. The brief warns that a polished interface alone does not establish the claim.

Checkpoint schedule

49 hours, six scored moments, roughly eight hours apart. Each wants a 60-second video plus the repo link. A missed checkpoint scores zero, and the average of the five check-ins is half the placement score — these are scored pitches, not progress reports.

Hour	When (ET / PT)	Dean	Chase
0	Sat 12:00 PM / 9:00 AM	Kickoff. Schema on a whiteboard, then scaffold. Together.	
8	Sat 8:00 PM / 5:00 PM	Estimator v0, corruption injector, Kalman baseline on the same data	Repo scaffold, one deck, fire spread, clean readings out
16	Sun 4:00 AM / 1:00 AM	Physical consistency check, ambiguity as sets not points	Multi-deck stacking, vertical conduction, first render
24	Sun 12:00 PM / 9:00 AM	Eval harness, four metrics, freeze the method	Scenario picker, chaos panel wired to corruption
32	Sun 8:00 PM / 5:00 PM	Allocator on belief, joint suppression + observability, hedging	Drone rendering, truth/belief/diff split view
40	Mon 4:00 AM / 1:00 AM	Identifiability result, briefings	Split-screen baseline, containment counter, polish. Stretch: high-rise structure plan
49	Mon 1:00 PM / 10:00 AM	Final: repo, writeup, live demo. README and rehearsal in this block.	

Two checkpoints land overnight — hour 16 and hour 40. On Pacific time those are 1:00 AM rather than 4:00 AM, which is a real structural advantage over East Coast teams. Decide now who covers which, and have the other person asleep.

What each checkpoint is for

Check-in 1 • hour 8

Sat 8:00 PM ET / 5:00 PM PT — the question

An ugly end-to-end loop: one deck, fire spreads, sensors report, estimator outputs something, baseline running alongside. No styling.

The video states the *question*, not the product. At this hour a top Innovation score means a breakthrough concept, not a finished build — so sell the insight that the fire destroys its own sensors.

Check-in 2 • hour 16

Sun 4:00 AM ET / 1:00 AM PT — first evidence

Consistency check working. Ambiguity detection outputting sets. Multi-deck with vertical conduction.

The video shows one number: our false certainty against the baseline's on one corruption case. First real proof the idea works.

Check-in 3 • hour 24

Sun 12:00 PM ET / 9:00 AM PT — the method is frozen

Eval harness sweeping corruption location, correlation structure, and k. All four metrics reported.

Freeze here. The brief requires evaluating against failures chosen after the method is fixed, so tuning past this point invalidates the result and a judge will ask when we froze.

The video says what surprised us. The brief explicitly asks what changed in our understanding, and almost no team will actually do that.

Check-in 4 • hour 32

Sun 8:00 PM ET / 5:00 PM PT — the allocator

Joint suppression-and-observability allocation on top of the belief.

The visible behavior: when uncertain between two compartments, the brain *hedges* and splits units to cover both. That is the money shot, because it is uncertainty turning into action a judge can watch.

Check-in 5 • hour 40

Mon 4:00 AM ET / 1:00 AM PT — the negative result

Two states we provably cannot distinguish, the operational cost, and the fix. Split-screen baseline, containment counter.

Strongest single moment of the event. Protect time for it.

Stretch if the result lands early: load a high-rise structure plan and run the same brain on it. "Same system, different building, nothing retrained" is a twenty-second beat that converts the whole market slide.

Final • hour 49

Mon 1:00 PM ET / 10:00 AM PT

Five-minute demo: problem, question, live run, break it, negative result, handoff and business case, then limits and next test — stated by us before a judge can ask.

Hard rules

- **Close the loop by hour 6.** If world, corruption, and brain are not composing before check-in 1, the check-in average is already lost.
- **Freeze the method at hour 24.** Non-negotiable; the validity of every number afterward depends on it.
- **Screen-record from hour 16 onward.** Every checkpoint needs footage and the final needs a backup.
- **Through hour 24, the estimator beats the visuals.** Every time.
- **Nothing in the final demo depends on a live API call.** Tech-stack access shuts off at the deadline — cache the generated plan and briefings.

Cut list, in order

When time runs short, cut from the top. Nothing lower goes before anything higher.

- **Cut first:** physics-simulated flight clip, Blender assets (generate geometry procedurally), speaker and egress drones, thermal camera input.
- **Cut second:** LLM briefings (fall back to templated text), 3D containment shell, commander override input.
- **Stretch, not cut:** the high-rise structure plan. It is the cheapest possible proof of generality once the pipeline works, so it goes above the cut line — but only after the identifiability result is in hand.
- **Never cut:** estimator, corruption model, baseline comparison, eval harness with the four metrics, one identifiability result, checkpoint videos.